实验一 实现单处理机下的进程调度程序

1. 程序中使用的数据结构及符号说明

- **Process 结构体**：表示进程控制块（PCB），包含以下成员：
  - `pid`： 进程号，类型为 `int`。
  - `arrival`： 进程到达时间，类型为 `int`。
  - `burst`： 进程要求的运行时间，类型为 `int`。
  - `priority`： 进程的初始优先级，类型为 `int`。
  - `ts`： 时间片大小，类型为 `int`。
  - `remaining`： 进程的剩余运行时间，类型为 `int`，在模拟过程中使用。
  - `start`： 当前调度段的开始时间，类型为 `int`，用于合并调度段。
- **Segment 结构体**：表示调度段，包含以下成员：
  - `order`： 调度顺序，类型为 `int`。
  - `pid`： 进程号，类型为 `int`。
  - `start`： 开始运行时间，类型为 `int`。
  - `end`： 结束运行时间，类型为 `int`。
  - `printPriority`： 调度时输出的优先级，类型为 `int`。

2. 各种调度算法的处理流程，重要模块的详细设计及功能和接口说明

2.1 FCFS（先来先服务）

- **处理流程**：
  - 按进程的到达时间和进程号进行排序。
  - 依次调度每个进程：如果当前时间早于进程的到达时间，则将时间快进到该进程的到达时间。
  - 每个进程运行至完成，记录调度段。
- **重要模块**：
  - 进程排序：使用 `std::sort` 函数，按到达时间和进程号升序排序。
  - 时间快进机制：确保调度时间不早于进程到达时间。
- **功能**：模拟先来先服务调度算法，生成调度段列表。

2.2 SJF（短作业优先，非抢占式）

- **处理流程**：

    - 按到达时间、运行时间和进程号排序。
    - 在每个时间点，从已到达且未完成的进程中选择运行时间最短的进程进行调度。
    - 选中的进程运行至完成，记录调度段。
- **重要模块**：
    - 选择运行时间最短的进程：若运行时间相等，则选择进程号较小的进程。
- **功能**：模拟非抢占式短作业优先调度算法，生成调度段列表。

2.3 SRT（最短剩余时间优先，抢占式）

- **处理流程**：
    - 按到达时间、运行时间和进程号排序。
    - 在每个时间点，选择已到达且未完成的进程中剩余运行时间最短的进程进行调度。
    - 若有新进程到达或当前进程不再是最优，则进行进程切换，记录调度段。
- **重要模块**：
    - 选择剩余运行时间最短的进程。
    - 处理进程切换：当有更优进程到达时，切换到新进程。
- **功能**：模拟抢占式最短剩余时间优先调度算法，生成调度段列表。

2.4 RR（时间片轮转）

- **处理流程**：
    - 按到达时间排序，使用队列管理就绪进程。
    - 每次从队首取进程运行一个时间片（或剩余时间），若未完成则放回队尾。
    - 新进程按到达顺序插入队尾。
- **重要模块**：
    - 队列管理：使用 `std::queue` 容器管理就绪进程。
    - 时间片控制：确保每个进程运行不超过时间片。
- **功能**：模拟时间片轮转调度算法，生成调度段列表。

2.5 DP（动态优先级）

- **处理流程**：
    - 按到达时间和进程号排序。
    - 在每个调度点，选择已到达且未完成的进程中优先级最小的进程进行调度。
    - 调度后，选中进程的优先级加3，其他未完成进程的优先级减1（不低于0）。
- **重要模块**：
    - 优先级更新机制：根据调度情况动态调整进程优先级。
    - 选择优先级最小的进程：若优先级相同，选择到达时间早的进程。
- **功能**：模拟动态优先级调度算法，生成调度段列表。

3. 源程序并附注释

```
#include <iostream>
#include <sstream>
#include <vector>
#include <queue>
#include <algorithm>
#include <string>
#include <climits>
using namespace std;

struct Process
{
    int pid;
    int arrival;
    int burst;    // 总运行时间
    int priority; // 初始优先级（或动态优先级初值）
    int ts;       // 时间片（调度算法4和5使用）

    // 以下用于模拟过程中使用
    int remaining; // 剩余执行时间
    int start;     // 当前段开始时间（用于合并段）
};

struct Segment
{
    int order;
    int pid;
    int start;
    int end;
    int printPriority; // 调度时输出的优先级
};

// -------------------- FCFS --------------------
// 先来先服务：按到达时间排序后，若当前时间小于进程到达时间则快进时间，调度后输出原始优先级
vector<Segment> fcfs(vector<Process> processes)
{
    // 按到达时间、pid排序
    sort(processes.begin(), processes.end(), [](const Process &a, const Process &b)
         { return a.arrival == b.arrival ? a.pid < b.pid : a.arrival < b.arrival; });
    vector<Segment> segments;
    int time = 0, order = 1;
    for (auto &p : processes)
    {
        if (time < p.arrival)
```

```
            time = p.arrival;
        int endtime = time + p.burst;
        segments.push_back({order++, p.pid, time, endtime, p.priority});
        time = endtime;
    }
    return segments;
}

// -------------------- SJF --------------------
// 非抢占式短作业优先：在所有已到达且未完成的进程中选择 cost（burst）最小的
vector<Segment> sjf(vector<Process> processes)
{
    int n = processes.size();
    vector<bool> done(n, false);
    int completed = 0, order = 1;
    vector<Segment> segments;
    // 为方便，记录原始索引顺序（排序后下标发生变化）
    vector<Process> procs = processes;
    // 按到达时间、burst、pid排序
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b)
         {
        if(a.arrival == b.arrival) {
            if(a.burst == b.burst)
                return a.pid < b.pid;
            return a.burst < b.burst;
        }
        return a.arrival < b.arrival; });
    int time = procs[0].arrival;
    while (completed < n)
    {
        // 找出所有已到达且未完成的进程
        vector<int> avail;
        for (int i = 0; i < n; i++)
        {
            if (!done[i] && procs[i].arrival <= time)
                avail.push_back(i);
        }
        if (avail.empty())
        {
            // 没有进程到达，快进到下一个未完成进程的到达时间
            for (int i = 0; i < n; i++)
            {
                if (!done[i])
                {
                    time = procs[i].arrival;
                    avail.push_back(i);
```

```
                break;
            }
        }
    }
    // 从 avail 中选出 burst 最小的（若相等则 pid 小者）
    int mini = avail[0];
    for (int idx : avail)
    {
        if (procs[idx].burst < procs[mini].burst ||
            (procs[idx].burst == procs[mini].burst && procs[idx].pid < procs[mini].pid))
            mini = idx;
    }
    int start = time;
    int endtime = time + procs[mini].burst;
    segments.push_back({order++, procs[mini].pid, start, endtime, procs[mini].priority});
    time = endtime;
    done[mini] = true;
    completed++;
}
return segments;
}

// -------------------- SRT --------------------
// 抢占式短作业优先：按单位时间模拟，选出剩余时间最短的运行，若有切换则记录上一个段
vector<Segment> srt(vector<Process> processes)
{
    int n = processes.size();
    vector<bool> done(n, false);
    // 为模拟方便，复制一份并初始化 remaining
    vector<Process> procs = processes;
    for (int i = 0; i < n; i++)
    {
        procs[i].remaining = procs[i].burst;
    }
    // 按到达时间、burst、pid排序
    sort(procs.begin(), procs.end(), [](const Process &a, const Process &b)
         {
        if(a.arrival == b.arrival) {
            if(a.burst == b.burst)
                return a.pid < b.pid;
            return a.burst < b.burst;
        }
        return a.arrival < b.arrival; });
    int time = procs[0].arrival;
    int completed = 0, order = 1;
    int current = -1; // 当前正在运行进程的下标
```

```
int segStart = time;
vector<Segment> segments;
while (completed < n)
{
    // 找出所有已到达且未完成的进程
    int mini = -1;
    int minRem = INT_MAX;
    for (int i = 0; i < n; i++)
    {
        if (!done[i] && procs[i].arrival <= time && procs[i].remaining < minRem)
        {
            minRem = procs[i].remaining;
            mini = i;
        }
    }
    if (mini == -1)
    {
        // 没有可运行进程，快进时间
        for (int i = 0; i < n; i++)
        {
            if (!done[i])
            {
                time = procs[i].arrival;
                break;
            }
        }
        continue;
    }
    // 若切换进程，则记录上一个进程的段
    if (current != mini)
    {
        if (current != -1 && segStart < time)
        {
            segments.push_back({order++, procs[current].pid, segStart, time, procs[current
        }
        current = mini;
        segStart = time;
    }
    // 决定运行时间：到下一个进程到达时刻或运行完当前进程
    int nextArrival = INT_MAX;
    for (int i = 0; i < n; i++)
    {
        if (!done[i] && procs[i].arrival > time && procs[i].arrival < nextArrival)
            nextArrival = procs[i].arrival;
    }
    int runTime = (nextArrival == INT_MAX) ? procs[mini].remaining : min(procs[mini].remai
```

```
        if (runTime <= 0)
            runTime = procs[mini].remaining;
        procs[mini].remaining -= runTime;
        time += runTime;
        if (procs[mini].remaining == 0)
        {
            segments.push_back({order++, procs[mini].pid, segStart, time, procs[mini].priority
            done[mini] = true;
            completed++;
            current = -1;
        }
    }
    return segments;
}

// -------------------- RR --------------------
// 时间片轮转：按到达顺序进入队列，每次取队首进程运行一个时间片（或剩余时间更短）
vector<Segment> rr(vector<Process> processes)
{
    int n = processes.size();
    // 按到达时间排序
    sort(processes.begin(), processes.end(), [](const Process &a, const Process &b)
         { return a.arrival < b.arrival; });
    // 初始化 remaining 为 burst
    for (auto &p : processes)
        p.remaining = p.burst;
    queue<Process *> q;
    int time = processes[0].arrival;
    int order = 1;
    int idx = 0; // 扫描 processes 的下标
    vector<Segment> segments;
    // 将所有到达时间 <= time 的进程入队
    while (idx < n && processes[idx].arrival <= time)
    {
        q.push(&processes[idx]);
        idx++;
    }
    while (!q.empty())
    {
        Process *cur = q.front();
        q.pop();
        if (time < cur->arrival)
            time = cur->arrival;
        if (cur->remaining > cur->ts)
        {
            segments.push_back({order++, cur->pid, time, time + cur->ts, cur->priority});
```

```
            time += cur->ts;
            cur->remaining -= cur->ts;
            // 将新到达的进程入队
            while (idx < n && processes[idx].arrival <= time)
            {
                q.push(&processes[idx]);
                idx++;
            }
            q.push(cur);
        }
        else
        {
            segments.push_back({order++, cur->pid, time, time + cur->remaining, cur->priority}
            time += cur->remaining;
            cur->remaining = 0;
            while (idx < n && processes[idx].arrival <= time)
            {
                q.push(&processes[idx]);
                idx++;
            }
        }
    }
    return segments;
}

// -------------------- DP --------------------
// 动态优先级调度：
// 1. 进程按到达时间排序；
// 2. 对于所有已到达进程（下标小于 i），若其到达时间在上次调度时间 last 与当前时间之间，则将其优先级
// 3. 在已到达且未完成进程中选择 priority 最小者；
// 4. 如果选中进程的剩余运行时间 <= 时间片，则调度完毕，否则运行一个时间片；
// 5. 调度后对选中进程优先级加 3，其它未完成进程优先级减 1（但不低于 0）。
vector<Segment> dp(vector<Process> processes)
{
    int n = processes.size();
    // 按到达时间、pid 升序排序
    sort(processes.begin(), processes.end(), [](const Process &a, const Process &b)
         { return a.arrival == b.arrival ? a.pid < b.pid : a.arrival < b.arrival; });
    // 初始化 remaining 为 burst
    for (int i = 0; i < n; i++)
    {
        processes[i].remaining = processes[i].burst;
    }
    vector<bool> done(n, false);
    int time = processes[0].arrival;
    int last = time;
```

```
int completed = 0;
int order = 1;
vector<Segment> segments;
while (completed < n)
{
    // 计算所有已到达进程个数 i，同时更新那些在 (last, time) 内到达的进程优先级减 1（若 > 1，否
    int i = 0;
    for (; i < n && processes[i].arrival <= time; i++)
    {
        if (last < processes[i].arrival && processes[i].arrival < time)
        {
            if (processes[i].priority > 1)
                processes[i].priority--;
            else
                processes[i].priority = 0;
        }
    }
    // 在已到达的进程中选出未完成且 priority 最小的
    int mini = -1;
    int minPriority = INT_MAX;
    for (int j = 0; j < i; j++)
    {
        if (!done[j] && processes[j].priority < minPriority)
        {
            minPriority = processes[j].priority;
            mini = j;
        }
    }
    if (mini == -1)
    {
        // 没有可调度进程，跳到下一个进程到达时刻
        if (i < n)
            time = processes[i].arrival;
        continue;
    }
    last = time;
    int runTime = 0;
    // 若剩余时间 <= 时间片，则调度完，否则运行一个时间片
    if (processes[mini].remaining <= processes[mini].ts)
    {
        runTime = processes[mini].remaining;
        segments.push_back({order++, processes[mini].pid, time, time + runTime, processes[
        time += runTime;
        processes[mini].remaining = 0;
        done[mini] = true;
        completed++;
```

```
        }
        else
        {
            runTime = processes[mini].ts;
            segments.push_back({order++, processes[mini].pid, time, time + runTime, processes[r
            time += runTime;
            processes[mini].remaining -= runTime;
        }
        // 调度后更新所有已到达且未完成的进程优先级：
        // 若为选中进程则加 3，否则减 1（但不低于 0）
        for (int j = 0; j < i; j++)
        {
            if (!done[j])
            {
                if (j == mini)
                    processes[j].priority += 3;
                else
                {
                    if (processes[j].priority > 1)
                        processes[j].priority--;
                    else
                        processes[j].priority = 0;
                }
            }
        }
    }
    return segments;
}

// -------------------- main --------------------
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int dispatchType;
    cin >> dispatchType;
    cin.ignore(); // 忽略行末换行

    vector<Process> processes;
    string line;
    while (getline(cin, line))
    {
        if (line.empty())
            continue;
        // 输入格式：pid/arrival/burst/priority/ts
```

```
        Process p;
        char slash;
        stringstream ss(line);
        ss >> p.pid >> slash >> p.arrival >> slash >> p.burst >> slash >> p.priority >> slash
        processes.push_back(p);
    }

    vector<Segment> segments;
    switch (dispatchType)
    {
    case 1:
        segments = fcfs(processes);
        break;
    case 2:
        segments = sjf(processes);
        break;
    case 3:
        segments = srt(processes);
        break;
    case 4:
        segments = rr(processes);
        break;
    case 5:
        segments = dp(processes);
        break;
    default:
        break;
    }

    // 输出所有调度段，每段格式：order/pid/start/end/printPriority
    for (auto &seg : segments)
    {
        cout << seg.order << "/" << seg.pid << "/" << seg.start << "/" << seg.end << "/" << se
    }

    return 0;
}
```

4. 采用的测试方法，对测试结果的分析，以及错误的分析

- **测试方法**：
  - 设计多种进程集，包括不同到达时间、运行时间、优先级和时间片的组合。

    - 分别对五种调度算法进行测试，验证调度顺序、开始和结束时间以及优先级变化。
- **测试结果分析**：
    - 验证 FCFS 算法是否按到达顺序调度。
    - 验证 SJF 算法是否选择运行时间最短的进程。
    - 验证 SRT 算法是否在有更短剩余时间的进程到达时进行抢占。
    - 验证 RR 算法是否按时间片轮转调度。
    - 验证 DP 算法是否正确更新优先级并选择优先级最高的进程。
- **错误分析**：
    - 若结果与预期不符，检查排序条件、时间快进逻辑和优先级更新机制。
    - 确保在抢占式算法中正确处理进程切换。

5. 本次实验经验及体会

通过本次实验，我深入理解了操作系统中进程调度的基本原理和实现方法。实现五种调度算法让我认识到不同策略的优缺点，例如 FCFS 简单但可能导致长作业阻塞，SRT 更灵活但计算复杂。特别是在动态优先级调度中，我体会到了优先级管理的挑战性。通过调试和验证，我提高了对算法设计和实现的掌握能力，也更加认识到测试的重要性。